

SUPSI

Valutazione di WebGPU per applicazioni di Realtà Virtuale tramite integrazione con OpenXR/OpenVR

Studente/i

Mazza Luca

Relatore

Peternier Achille

Correlatore

Paoliello Marco

Committente

-

Corso di laurea

**Ingegneria informatica
(Informatica TP)**

Modulo

C11287

Anno

2025 / 2026

Data

27 maggio 2026

*Vorrei qui ringraziare Luca
per l'incredibile impegno nello sviluppo di questa tesi*

Indice

1 Abstract	1
2 Introduzione	3
2.1 Pianificazione	4
2.2 Requisiti	5
3 Stato dell'arte	6
4 Design e implementazione	7
5 Test e validazione	8
5.1 Risultati	8
6 Risultati	9
6.1 Manuale d'uso	9
7 Conclusioni	10
7.1 Sviluppi futuri	10
Glossario	I

Elenco delle figure

Elenco delle tabelle

2.1	Requisiti del progetto	5
-----	----------------------------------	---

Capitolo 1

Abstract

Capitolo 2

Introduzione

Negli ultimi anni, l'evoluzione delle API grafiche ha portato ad un progressivo allontanamento dagli standard tradizionali, come OpenGL¹, in favore di soluzioni più moderne, che forniscono più controllo direttamente sull'hardware delle unità grafiche dei calcolatori. In questo panorama, WebGPU² si è imposta non solo come nuova generazione di API grafiche per il Web, ma anche come una solida alternativa per contesti nativi, grazie ai kit di sviluppo che la rendono compatibile con diversi linguaggi a basso livello, come C++ e Rust. Un progetto precedentemente sviluppato ha già analizzato e dimostrato l'efficacia della transizione da OpenGL a WebGPU per corsi introduttivi alla computer grafica.

Lo scopo del presente progetto è di estendere tale studio di fattibilità in un settore con ancora maggiori esigenze, ovvero il dominio della *Realtà Virtuale* (VR). L'obiettivo principale è la valutazione dell'impiego di WebGPU in un ambiente nativo C/C++, e che questo possa rappresentare una valida alternativa ad OpenGL per lo sviluppo di applicazioni VR, investigando la complessità e l'efficacia della sua integrazione con framework standard quali OpenXR³ e OpenVR⁴.

Come per quanto riguarda i progetti precedentemente sviluppati, per valutare correttamente questa tecnologia è fondamentale tener sempre presente il contesto accademico in cui si andrà ad operare. Questo lavoro si rivolge in primis al modulo opzionale M-I6331Z - *Realtà Virtuale* della SUPSI, che si basa sulle conoscenze acquisite nel corso obbligatorio M-I5203 - *Grafica*. Lo scopo di tale corso è di insegnare allo studente lo sviluppo di un'applicazione interattiva in C++, interfacciandosi direttamente con le API di basso livello, senza affidarsi a game engine preconfezionati.

In questo scenario, la semplicità d'utilizzo è cruciale nel determinare la fattibilità; una soluzione troppo complessa rischierebbe di spostare inavvertitamente il focus del corso. Gli studenti e il docente si ritroverebbero a dedicare troppo tempo a districarsi tra le difficoltà di implementazione dell'API grafica, sottraendolo all'apprendimento dei concetti fondamentali della *Realtà Virtuale*. Tuttavia, lo sviluppo VR impone requisiti stringenti in termini di latenza, prestazioni e gestione diretta dell'hardware, elementi dove le API moderne come WebGPU eccellono, grazie al loro paradigma basato su pipeline esplicite.

Durante il progetto verrà pertanto sviluppato un prototipo funzionante che utilizzi WebGPU come backend grafico per il rendering di una scena VR stereoscopica. Verrà creato un layer di integrazione tra WebGPU e le API VR, affrontando e risolvendo le sfide tecniche legate all'acquisizione delle pose dei dispositivi, alla gestione delle swapchains e alla rigorosa sincronizzazione dei frame richiesta dal runtime, come ad esempio SteamVR.

¹<https://opengl.org>

²<https://webgpu.org>

³<https://khronos.org/openxr>

⁴<https://github.com/ValveSoftware/openvr>

L'esito di questa implementazione permetterà infine di confrontare l'approccio WebGPU con le soluzioni tradizionali, misurando in modo oggettivo il livello di effort richiesto per lo sviluppo e le performance ottenute. I risultati raccolti forniranno delle linee guida concrete per determinare l'effettiva fattibilità e le modalità ottimali per l'adozione di WebGPU in ambito didattico e applicativo nel contesto della Realtà Virtuale.

2.1 Pianificazione

Di seguito sono descritte le differenti fasi atte allo sviluppo del progetto. In quanto la natura del progetto sia sperimentale, ed infine l'obiettivo sia quello di verificare la fattibilità di utilizzo, il piano di lavoro non eccede nei dettagli o nei vincoli ma risulta completo, garantendo la flessibilità necessaria e l'immediata risposta ad eventuali problemi o deviazioni dai mezzi discussi inizialmente.

1. **Analisi WebGPU:** Analizzare l'approccio necessario per lo sviluppo di un'applicazione grafica facente uso dell'API di WebGPU.
2. **Analisi OpenXR:** Analizzare l'approccio utilizzato nei progetti precedenti (specialmente per quanto riguarda *XRBridge*⁵) per lo sviluppo di un'applicazione grafica che faccia utilizzo di OpenXR per fornire funzionalità per la realtà virtuale.
3. **Demo WebGPU indipendente:** Derivante dall'analisi riguardante l'API di WebGPU, sviluppare una demo che possa rappresentare un punto di partenza per lo sviluppo della demo finale. Questa deve concentrarsi sulle funzionalità dell'API WebGPU. Questa demo deve rimanere semplice e ridurre la complessità di integrazione di ulteriori funzionalità, specialmente quelle riguardanti la realtà virtuale.
4. **Demo OpenXR indipendente:** Derivante dall'analisi riguardante OpenXR, sviluppare una demo che possa rappresentare un punto di partenza per lo sviluppo della demo finale. Questa deve concentrarsi sulle funzionalità di realtà virtuale. Questa demo deve essere imperativamente concentrata sui requisiti del corso di realtà virtuale e concentrarsi ad adattare unicamente ciò che è necessario per il corso. Ulteriormente, questa deve essere integrabile nell'ambiente WebGPU, deve quindi avere un approccio modulare, il quale permetterà di rimuovere le funzionalità OpenGL e sostituirle con quelle di WebGPU.
5. **Swapchain condivisa:** Sviluppare una swapchain, condivisa tra WebGPU e OpenXR, che permetta di condividere le texture da mostrare tramite gli occhi dell'headset, tra l'API WebGPU e quella di OpenXR. Ciò si riduce nella produzione di una libreria bridge, che utilizzi uno dei "minimi comun denominatori"⁶ tra le due tecnologie, in questo caso una tra Vulkan, DirectX o Metal.
6. **Sincronizzazione & Rendering Stereoscopico:** Integrare la sincronizzazione del loop di rendering di WebGPU con i tempi dettati da OpenXR. Deve essere inoltre modificata la pipeline di WebGPU per renderizzare la scena due volte, basandosi sulle matrici di *projection* e *view* fornite da OpenXR per occhio sinistro e destro.
7. **Demo unificata:** Produrre una demo che unisca tutte le funzionalità richieste dal corso di Realtà Virtuale, utilizzando l'architettura prodotta nelle fasi precedenti. Questa deve dimostrare in maniera semplice e efficace i risultati ottenibili dalla combinazione di WebGPU e OpenXR.

⁵<https://github.com/JollyCookie2000/xr-bridge>

⁶WebGPU si interfaccia direttamente con l'API di sistema per la rappresentazione nativa di immagini grafiche.

8. **Benchmarking:** Misurare il Framerate (FPS) e la latenza della demo, utilizzando appropriati strumenti di misurazione.
9. **Valutazione effort:** Confrontare la pipeline prodotta con quelle utilizzate negli anni passati del corso; valutare le principali differenze e le difficoltà di comprensione, la verbosità e l'osticita di debug.
10. **Linee guida conclusive:** Produrre una guida finale che mostri brevemente le conclusioni tratte durante lo sviluppo e che guidi l'eventuale integrazione della pipeline nel corso.

Questo rapporto è stato scritto durante tutta la durata del progetto, parallelamente a tutte le fasi descritte sopra.

2.2 Requisiti

ID	Descrizione	Note
R-01		

Tabella 2.1: Requisiti del progetto

Capitolo 3

Stato dell'arte

Capitolo 4

Design e implementazione

Capitolo 5

Test e validazione

5.1 Risultati

Capitolo 6

Risultati

6.1 Manuale d'uso

Capitolo 7

Conclusioni

7.1 Sviluppi futuri

Glossario

DirectX (in origine chiamato "Game SDK") è una raccolta di API per lo sviluppo semplificato di videogiochi per sistema operativo Microsoft Windows. Il componente DirectX Graphics permette la presentazione di grafica 2D e 3D a video, direttamente interfacciandosi con la GPU. 4

Framerate (FPS) Frequenza di cattura o riproduzione dei fotogrammi che compongono un filmato. Un filmato, o un'animazione al computer, è infatti una sequenza di immagini riprodotte ad una velocità sufficientemente alta da fornire, all'occhio umano, l'illusione del movimento. 5

Metal Insieme di API grafiche e di calcolo a basso livello, sviluppate da Apple per offrire accesso diretto alla GPU dei suoi dispositivi. 4

OpenVR API e Runtime che consente accesso alle risorse hardware VR di diversi produttori, senza richiedere conoscenze specifiche dell'hardware stesso. 3

OpenXR Standard aperto che fornisce un set di API per lo sviluppo di applicazioni XR (realtà virtuale, mista, ecc...) compatibili con un grande range di dispositivi AR e VR. 3

Vulkan Interfaccia programmatica di applicazione (API) di basso livello, multi-piattaforma in 2D e 3D, annunciata la prima volta al GDC 2015 da Khronos Group. Inizialmente venne presentata come "OpenGL di prossima generazione". 4

